

1. [Data modeling and constraints, 25 points] Consider the following tables for managing courses in Fall 2025. Note that **primary key attributes are underlined** and **stated**. For each table, some sample data are also provided.

- Student(sid, name, program)
  - Primary key: sid.
  - Note that sid is student id.
  - Sample data: (100, 'John', 'Data Science'), (200, 'David', 'CS').
- Course(cno, title, unit)
  - Primary key: cno.
  - Note that cno is course number.
  - Sample data: ('DSCI 551', 'Database', 4), ('DSCI 552', 'Machine learning', 4), ('DSCI 553', 'Data mining', 4).
- Takes(sid, cno)
  - Primary key: (sid, cno).
  - Sample data: (100, 'DSCI 551'), (100, 'DSCI 552'), (200, 'DSCI 552')
- TA(sid, hourly\_rate)
  - Primary key: sid.
  - Sample data: (200, 20), i.e., TA whose sid is 200 and is paid with \$20/hour.
- Assists(sid, cno, hours)
  - Primary key: (sid, cno).
  - Note that "hours" refers to the number of hours a TA works for a particular course.
  - Sample data: (200, 'DSCI 551', 10), (200, 'DSCI 553', 10), (100, 'DSCI 553', 8).

a. [15 points] Reverse engineer the above tables into its corresponding ER model. Be sure to properly state the entities, relationships (including the multiplicity), attributes, and keys. For subclass, state key attributes and attributes unique to the subclass entity; **no** need to state inherited attributes.

Entities:

- Student
  - attributes: sid, name, program; key: sid
- Course
  - attributes: cno, title, unit; key: cno

- o TA
  - Attributes: sid, hourly\_rate; key: sid

Relationships:

- o Takes: many to many between Student and Course
- o Is-a: from TA to Student (superclass)
- o Assists: many to many between TA and Course; attribute: hours.

Rubrics:

Entity (3 points):

-1 point for each wrong entity

Attribute (6 points):

For each entity: -1 point for one wrong attribute

-2 points if more than one attributes are wrong

Relationship (6 points, 2 points for each relationship):

-1 point for each wrong relationship

-1 point for each wrong multiplicity, if subclass, -1 point for wrong key attributes and attributes

- b. [4 points] Write a "create table" statement for the table **Takes**. Be sure to properly declare primary key and foreign keys.

```
Create table Takes (sid int, cno varchar(20),
                  Primary key (sid, cno),
                  Foreign key (sid) references Student(sid),
                  Foreign key (cno) references Course(cno));
```

Rubrics:

-1 point for each small mistake

- c. [6 points] Explain what operations (insert/delete/update) to which tables might cause possible violations of FK constraints declared in the table **Takes**.

```
Student: delete, update
Course: delete, update
Takes: insert, update
```

Rubrics:

For each table, -1 point is wrong operation or missing one operation

2. [SQL, 25 points] Consider the same tables as in the previous question, reproduced here:

- Student(sid, name, program)
- Course(cno, title, unit)
- Takes(sid, cno)
- TA(sid, hourly\_rate)
- Assists(sid, cno, hours)

Write an SQL query for each of the following questions. 5 points each question.

- a. Find out how many students there are in each program. Order the output in the descending order of the number of students.

```
Select program, count(*) cnt
From Student
Group by program
Order by cnt desc;
```

Rubrics:

-1 point for each small mistake

- b. Find out how many students take both DSCI 551 and DSCI 552.

```
Select count(*) AS num_students
From (
  Select sid
  From Takes
  Where cno in ('DSCI 551', 'DSCI 552')
  Group by sid
  Having count(distinct cno) = 2
) as t;
```

Rubrics:

-1 point for each small mistake

- c. Find out names of TAs who assist DSCI 551 and are paid at least \$20 a hour.

```
Select name
From student natural join Assists natural join TA
```

Where assists.cno = 'DSCI 551' and TA.hourly\_rate >= 20;

Rubrics:

-1 point for each small mistake

- d. Find out which courses have the largest number of enrollments. Note that there may be multiple such courses. Return course numbers.

```
with t as (  
    select cno, count(*) cnt  
    from Takes  
    group by cno)  
select cno from t where cnt = (select max(cnt) from t);
```

Rubrics:

-1 point for each small mistake

- e. Find out how many units John in the Data science program is taking. You can assume that there is only one student named "John" in the data science program.

```
select sum(unit)  
from Course natural join Takes natural join Student  
where name = "John" and program = 'Data science';
```

Rubrics:

-1 point for each small mistake

3. [MongoDB, 25 points] Now consider storing the tables in the previous question in MongoDB as collections. There is a collection for each table whose primary key attributes are stored in the `_id` field of the collection. Here are the collections and some sample data.

- Student:
  - `{_id: {sid: 100} , name: "John", program: "Data science"}`
  - `{_id: {sid: 200}, name: "David", program: "CS"}`
- Course:
  - `{_id: {cno: "DSCI 551"}, title: "Database", unit: 4}`

- { \_id: {cno: "DSCI 552"}, title: "Machine learning", unit: 4}
- { \_id: {cno: "DSCI 553"}, title: "Data mining", unit: 4}
- Takes:
  - { \_id: {sid: 100, cno: "DSCI 551"}}
  - { \_id: {sid: 100, cno: "DSCI 552"}}
  - { \_id: {sid: 200, cno: "DSCI 552"}}
  -
- TA:
  - { \_id: {sid: 200}, hourly\_rate: 20}
- Assists:
  - { \_id: {sid: 200, cno: "DSCI 551"}, hours: 10}
  - { \_id: {sid: 200, cno: "DSCI 553"}, hours: 10}
  - { \_id: {sid: 100, cno: "DSCI 553"}, hours: 8}

Write a MongoDB function for each of the following questions. 5 points each question.

- a. Find names of students in "CS" or "Data science" program. Return names only (excluding \_id too).

```
db.Student.find({program: {$in: ["Data science", "CS"]}, {name: 1, _id: 0})
```

Rubrics:

-1 point for each small mistake

- b. Find out how many students there are in each program. Order the output in the descending order of the number of students.

```
db.Student.aggregate({$group: {_id: "$program", cnt: {$sum: 1}}},
                     {$sort: {cnt: -1}})
```

Rubrics:

-1 point for each small mistake

- c. Find out courses that John in the Data science program is taking. Return course numbers only.

```
db.Student.aggregate([
  {$match: {name: "John", program: "Data science"}},
  {$lookup: {
```

```

    from: "Takes",
    localField: "_id.sid",
    foreignField: "_id.sid",
    as: "courses"}
  },
  { $unwind: "$courses" },
  { $project: { _id: 0, cno: "$courses._id.cno" }}
]);

```

Rubrics:

-1 point for each small mistake

- d. Find the top 3 courses ranked by the number of students enrolled in the courses. Return course numbers only.

```

db.Takes.aggregate({$group: {_id: "$_id.cno", cnt: {$sum: 1}}},
  {$sort: {cnt: -1}},
  {$limit: 3},
  {$project: {cnt: 0}})

```

Rubrics:

-1 point for each small mistake

- e. Change the program of the student whose sid is 100 from "Data science" to "CS" and add a new attribute "GPA" and set its value to 3.8. Note that you should **NOT** remove existing attributes of this student.

```

db.Student.updateOne({"_id.sid": 100}, {$set: {program: "CS", gpa: 3.8}})

```

Rubrics:

-1 point for each small mistake

4. [External sorting, 10 points] Consider sorting a table with 103 blocks (i.e.,  $B(R) = 103$ ) with a memory of 5 pages ( $M = 5$ ), using the external sorting algorithm discussed in class. The algorithm will use all the available memory in its sorting and merging phases.

- a. [3 points] How many runs will it generate in the sorting phase? What is the size of each run? Show your derivations.

$\text{Ceil}(103/5) = 21$  runs

First 20 runs: 5 blocks/run

Last run: 3 blocks

Rubrics:

- 1 point if no answer for first question
- 1 point if no answer for second question
- 1 point if correct answers but no derivations

- b. [2 points] How many ways of merging can the algorithm perform? Explain your answer.

4-way, 4 input buffer pages + 1 output buffer page = 5

Rubrics:

- 2 point for incorrect answer
- 1 point if correct answer but no explanation

- c. [3 points] How many runs will the first **merging** pass generate? What is the size of each run? Show your derivations.

$\text{Ceil}(21/4) = 6$  runs

first 5 runs:  $5 * 4 = 20$  blocks/run; last run: 3 blocks

Rubrics:

- 1 point if no answer for first question
- 1 point if no answer for second question
- 1 point if correct answer but no derivations

- d. [2 points] In addition to the first merging phase, how many **additional** merging passes are needed to completely sort the table? Explain your answer.

merging 2: # of runs:  $\text{ceiling}(6/4) = 2$

1. take first 4 runs  $\Rightarrow 20 * 4 = 80$  blocks

2. take last 2 runs  $\Rightarrow 20 + 3 = 23$  blocks

merging 3: take 2 runs  $\Rightarrow$  a single run

Thus, need 2 additional merging passes.

Rubrics:

- 2 point for incorrect answer
- 1 point if correct answer but no explanation

5. [Miscellaneous, 15 points]

- a. [3 points] What are the advantages and disadvantages of materialized views, compared to virtual views?

Adv: faster query response

Disadvantages: need data storage, might return stale data

Rubrics:

-1 point for incorrect advantage

-1 point for each incorrect disadvantage (total 2 points)

- b. [3 points] Consider a person table with an attribute on person age. Write a "create table" command that can limit the age values between 0 and 150.

Create table person(age tinyint unsigned check (age <= 150) or

Create table person(age tinyint check (age >= 0 and age <= 150))

Accept other integer data types (e.g., int), instead of tinyint

Rubrics:

-1 point for each small mistake

- c. [4 points] Which attributes in the following table are fixed length? Explain your answer.

```
CREATE TABLE Product (  
    pid int primary key,  
    name varchar(20),  
    description varchar(100),  
    brand char(20));
```

pid: int, 4 bytes always

brand: 20 bytes/characters, always (padding zeros if not enough)

name and description are both variable length, database only

stored actual characters.

Rubrics:

-1 point for missing one attribute or incorrect attribute (total 2 points)

-1 point if no explanation for each attribute (total 2 points)

- d. [5 points]

- i. [2 points] What is the purpose of the degree/order parameter of a B+-tree?

Keep the nodes in the tree balanced (avoid situation where some nodes store a lot of keys, while others few).

Rubrics:

-2 points for wrong answer

- ii. [3 points] Is it possible that inserting a new key into a B+-tree may increase the number of levels of the tree? Explain your answer.

Yes, the insertion may cause a (leaf) node to split and the split may propagate up to the root node, causing it to split too. Splitting the root node will create a new root node as the new parent of split-generated nodes. Hence, level increases by one.

Rubrics:

-1 point for "No"

-2 points if no explanation or incorrect explanation